Stop Collecting Everything

Anfängeranleitung: Baseline, Sampling und KI/MCP Sampling Advisor

Ziel dieser Anleitung

Du startest an dem Punkt, an dem der Astronomy Shop bereits läuft. Danach lernst du in drei klar getrennten Schritten: 1. eine belastbare 100%-Baseline aufzunehmen, 2. Sampling bewusst zu konfigurieren und zu messen und 3. den MCP Sampling Advisor mit einer KI zu nutzen, um datenbasierte Sampling-Empfehlungen zu erzeugen.

Wichtig: Diese Anleitung ist für jemanden geschrieben, der OpenTelemetry Sampling noch nie selbst konfiguriert hat. Jede Änderung wird mit Datei -> Stelle -> Änderung -> Neustart -> Kontrolle -> Messung erklärt.

Ausgangslage: Der funktionierende Stack wird mit compose.yaml, compose.observability.yaml, compose.extras.yaml und compose.podman.yaml gestartet. Prometheus ist unter http://localhost:9090 erreichbar; Grafana und Jaeger laufen hinter dem Frontend-Proxy.

0. Das mentale Modell

Bevor du etwas änderst, ist dieser Zusammenhang entscheidend:

```
Astronomy Shop
  | erzeugt Traces und Spans
  v
OpenTelemetry Collector
  |  <-- HIER konfigurieren wir Sampling
  +--> Span Metrics --> Prometheus --> MCP Server --> KI
  |
  +--> exportierte Traces --> Jaeger

KI-Empfehlung --> Human Review --> neue Sampling-Policy --> erneut messen
```

> **Merksatz**
>
> Du änderst für dieses Lab nicht den Shop. Du änderst die Trace-Verarbeitung im OpenTelemetry Collector. Die zentrale Datei ist src/otel-collector/otelcol-config.yml.

Span: einzelne Operation innerhalb eines Requests. Trace: zusammengehörige Kette von Spans. Probabilistic Sampling: behält nur einen Prozentsatz. Tail Sampling: wartet auf den Trace und trifft die Entscheidung anhand von Eigenschaften wie Fehler oder Latenz.

Schritt 1 - 100%-Baseline sauber ermitteln

Zweck: Du brauchst zuerst einen unveränderten Vergleichspunkt. Sonst kannst du später keine seriöse Reduktion zeigen.

1.1 In das Demo-Verzeichnis wechseln

```
cd ~/Documents/otel-cost-workshop/opentelemetry-demo
```

1.2 Collector-Konfiguration sichern

```
cp src/otel-collector/otelcol-config.yml \
  src/otel-collector/otelcol-config.original.yml
```

Diese Sicherung ist dein Rückweg. Für die Baseline fügst du noch keinen Sampler hinzu.

1.3 Prüfen, dass Shop und Observability laufen

```
podman ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"

# Collector auf Konfigurationsfehler prüfen
podman logs otel-collector 2>&1 | grep -iE "error|invalid|failed|unknown"
```

Wenn der grep-Befehl nichts ausgibt, ist das für diese Fehlersuche ein gutes Zeichen. Prüfe zusätzlich Shop, Jaeger und Prometheus im Browser.

1.4 Einen reproduzierbaren Test festlegen

Verwende für jede Variante denselben Traffic, dieselbe Laufzeit und denselben Messzeitraum. Führe dieselbe Shop-Journey mehrfach aus oder verwende den laufenden Load Generator. Ändere zwischen den Experimenten ausschließlich das Sampling.

1.5 Die tatsächlich vorhandenen Span-Metriken entdecken

```
curl -s 'http://localhost:9090/api/v1/label/__name__/values' | python3 -m json.tool

# In der Prometheus-UI zusätzlich suchen:
{__name__=~".*span.*calls.*"}
{__name__=~".*span.*duration.*"}
```

Warum dieser Schritt? Die konkreten Metriknamen können von der Demo-/Collector-Version abhängen. Verwende in den nachfolgenden Queries nur Namen, die in deiner Umgebung tatsächlich existieren.

1.6 Baseline messen

Wenn der Name traces_span_metrics_calls_total in deiner Umgebung existiert, kannst du für Volumen z. B. diese Query verwenden:

```
sum by (service_name, span_name)(
  rate(traces_span_metrics_calls_total[5m])
)
```

Für Fehlerquote und p95-Latenz enthält dein bisheriges Lab folgende Query-Ideen:

```
100 * sum by (service_name, span_name)(
  rate(traces_span_metrics_calls_total{status_code="STATUS_CODE_ERROR"}[5m])
) / sum by (service_name, span_name)(
  rate(traces_span_metrics_calls_total[5m])
)

histogram_quantile(0.95,
  sum by (le,service_name,span_name)(
    rate(traces_span_metrics_duration_milliseconds_bucket[5m])
  )
)
```

Notiere zusätzlich die Collector-Ressourcen:

```
podman stats --no-stream otel-collector
```

Was bedeutet 100%-Baseline?

Für dieses Experiment ist die Baseline die Collector-Konfiguration ohne die in den nächsten Schritten hinzugefügte Sampling-Policy. Sie ist der Vergleichswert für 50 %, 10 %, Tail und KI-Policy.

Messgröße	Baseline	50 %	10 %	Tail	KI-Policy
Testdauer					
Calls/Spans bzw. Rate					
Fehler sichtbar?					
Slow Traces sichtbar?					
Collector CPU/RAM					

Schritt 2 - Sampling einstellen, prüfen und vergleichen

Das wichtigste Prinzip

Ein Processor muss zweimal berücksichtigt werden: (1) unter processors: definieren und (2) in der service -> pipelines -> traces -> processors-Liste aktivieren. Nur definieren reicht nicht.

2.1 Datei öffnen

```
cd ~/Documents/otel-cost-workshop/opentelemetry-demo
nano src/otel-collector/otelcol-config.yml
```

Wenn du lieber VS Code verwendest, öffne dieselbe Datei dort. Suche zuerst nach processors:, danach nach service: und darunter pipelines: / traces:.

2.2 Experiment A: 50 % probabilistisches Sampling

A. Processor definieren. Unter dem vorhandenen Top-Level-Block processors: ergänzen:

```
processors:
  # ... vorhandene Processor-Konfiguration NICHT löschen ...

  probabilistic_sampler:
    sampling_percentage: 50
```

B. Processor in der Trace-Pipeline aktivieren. In der bereits vorhandenen Liste der Trace-Processor probabilistic_sampler ergänzen. Die vorhandenen Processor-Namen bleiben erhalten:

```
service:
  pipelines:
    traces:
      receivers: [otlp]
      processors: [resource_detection, memory_limiter,
        transform/sanitize_spans, gen_ai_normalizer,
        transform/redact_sensitive_data, redaction,
        probabilistic_sampler]
      exporters: [debug, span_metrics]
```

Die Namen oben entsprechen der Konfiguration aus deinem bisherigen Guide. Wenn deine lokale Datei inzwischen andere zusätzliche Processor enthält, nicht überschreiben, sondern nur den Sampler ergänzen.

2.3 Änderung laden

Eine gespeicherte YAML-Datei ändert einen bereits laufenden Collector nicht automatisch. Erstelle den Stack deshalb mit deinem funktionierenden Compose-Aufruf neu:

```
podman compose --project-name otel-demo \
  --env-file .env --env-file .env.override \
  -f compose.yaml -f compose.observability.yaml \
  -f compose.extras.yaml -f compose.podman.yaml \
  up --force-recreate --remove-orphans --detach
```

Danach prüfen:

```
podman compose --project-name otel-demo \
  -f compose.yaml -f compose.observability.yaml \
  -f compose.extras.yaml -f compose.podman.yaml ps

podman logs otel-collector 2>&1 | grep -iE "error|invalid|failed|unknown"
```

Wenn der Collector wegen YAML oder einem unbekannten Processor nicht startet, gehe nicht zum Messtest über. Korrigiere zuerst die Collector-Konfiguration.

2.4 50%-Test messen

Erzeuge exakt denselben Traffic wie bei der Baseline. Verwende denselben Query-Zeitraum. Trage die Werte in die 50-%-Spalte ein. Die Einzelmessung wird wegen probabilistischer Auswahl nicht zwangsläufig exakt 50 % der Baseline ergeben.

2.5 Experiment B: auf 10 % wechseln

Jetzt änderst du nur diese eine Zeile:

```
probabilistic_sampler:
  sampling_percentage: 10
```

Danach wieder: speichern -> Stack neu erstellen -> Collector-Logs prüfen -> identischen Traffic laufen lassen -> identische PromQL-Abfragen -> Werte in die 10-%-Spalte eintragen.

2.6 Experiment C: Tail Sampling

Für Tail Sampling entfernst du probabilistic_sampler sowohl aus seiner Definition als auch aus der Trace-Pipeline. Danach definierst du stattdessen den Tail-Sampler:

```
processors:
  # ... vorhandene Processor bleiben ...
  tail_sampling:
    decision_wait: 10s
    num_traces: 5000
    expected_new_traces_per_sec: 100
    policies:
      - name: keep-errors
        type: status_code
        status_code: {status_codes: [ERROR]}
      - name: keep-slow
        type: latency
        latency: {threshold_ms: 1000}
      - name: sample-normal
        type: probabilistic
        probabilistic: {sampling_percentage: 10}
```

In der Trace-Pipeline kommt nun tail_sampling an die Stelle des zuvor verwendeten probabilistic_sampler. Vorhandene Processor bleiben unberührt.

Starte den Stack erneut und teste drei Arten von Traffic: normal, Fehler und langsam. Warte bei der Auswertung mindestens die konfigurierte decision_wait: 10s ab.

Was du demonstrierst

Probabilistisches Sampling reduziert pauschal. Tail Sampling ermöglicht eine inhaltliche Entscheidung: Fehler und langsame Traces können vollständig erhalten bleiben, während normale Traces nur anteilig behalten werden.

Schritt 3 - MCP Sampling Advisor + KI

Jetzt kommt die eigentliche Optimierungsstory: Die KI soll nicht raten. Der MCP-Server liest Messdaten aus Prometheus und stellt strukturierte Werkzeuge bereit. Die KI bewertet diese Daten und formuliert eine Empfehlung. Änderungen am Collector bleiben eine menschliche Entscheidung.

3.1 Was der vorhandene MCP Advisor analysiert

MCP-Tool	Bedeutung
top_operations	Operationen mit hohem Call-Volumen
error_hotspots	Operationen mit auffälliger Fehlerrate
slow_operations	Operationen mit hoher p95-Latenz
sampling_candidates	führt Volumen, Fehler und Latenz für die Beratung zusammen

Die Identifikation basiert im vorhandenen Code auf service_name und span_name. Das ist keine vollständige automatische API-Inventarisierung, sondern die Analyse der Operationen, die deine Span Metrics sichtbar machen.

3.2 Vor MCP: echte Metrik- und Labelnamen bestätigen

```
curl -s 'http://localhost:9090/api/v1/label/__name__/values' | python3 -m json.tool
```

Bestätige Calls-Metrik, Duration-Bucket-Metrik sowie Service-, Span- und Status-Label. Falls die Namen von den Defaults abweichen, trägst du die echten Namen in .env.override ein.

3.3 Verzeichnis und Dateien anlegen

```
mkdir -p mcp-sampling-advisor
cd mcp-sampling-advisor
```

requirements.txt:

```
mcp[cli]>=2.2,<3
httpx>=0.27,<1
```

Containerfile:

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY server.py .
EXPOSE 8000
CMD ["python", "server.py"]
```

3.4 server.py erstellen

Lege mcp-sampling-advisor/server.py mit dem Code aus dem vorhandenen Lab an:

```
import os, httpx
from mcp.server import MCPServer
mcp=MCPServer("OTel Sampling Advisor")
PROM=os.getenv("PROM_URL","http://prometheus:9090")
CALLS=os.getenv("CALLS_METRIC","traces_span_metrics_calls_total")
DUR=os.getenv("DURATION_BUCKET_METRIC","traces_span_metrics_duration_milliseconds_bucket")
SVC=os.getenv("SERVICE_LABEL","service_name")
SPAN=os.getenv("SPAN_LABEL","span_name")
STATUS=os.getenv("STATUS_LABEL","status_code")
ERR=os.getenv("ERROR_VALUE","STATUS_CODE_ERROR")
async def q(expr):
  async with httpx.AsyncClient(timeout=20) as c:
    r=await c.get(f"{PROM}/api/v1/query",params={"query":expr})
    r.raise_for_status(); return r.json()["data"]["result"]
def out(rows):
  return [{"labels":r.get("metric",{}),"value":float(r["value"][1])} for r in rows]
@mcp.tool()
async def top_operations(window:str="5m",limit:int=15):
  return out(await q(f"topk({limit},sum by ({SVC},{SPAN})(rate({CALLS}[{window}])))"))
@mcp.tool()
async def error_hotspots(window:str="5m",limit:int=15):
  e=f'100*sum by ({SVC},{SPAN})(rate({CALLS}{{{STATUS}="{ERR}"}}[{window}]))/clamp_min(sum by ({SVC},{SPAN})(rate({CALLS}[{wind
  return out(await q(f"topk({limit},{e})"))
@mcp.tool()
async def slow_operations(window:str="5m",limit:int=15):
  e=f"histogram_quantile(0.95,sum by (le,{SVC},{SPAN})(rate({DUR}[{window}])))"
  return out(await q(f"topk({limit},{e})"))
@mcp.tool()
async def sampling_candidates(window:str="5m",limit:int=20):
  return {"volume":await top_operations(window,limit),"errors":await error_hotspots(window,limit),"slow":await slow_operations(window,limit),"ru
if __name__=="__main__":
  mcp.run("streamable-http",host="0.0.0.0",port=8000)
```

3.5 compose.mcp.yaml im Repository-Hauptverzeichnis erstellen

```
services:
  mcp-sampling-advisor:
    build:
      context: ./mcp-sampling-advisor
      dockerfile: Containerfile
    environment:
      PROM_URL: http://prometheus:9090
      CALLS_METRIC: ${CALLS_METRIC:-traces_span_metrics_calls_total}
      DURATION_BUCKET_METRIC: ${DURATION_BUCKET_METRIC:-traces_span_metrics_duration_milliseconds_bucket}
      SERVICE_LABEL: ${SERVICE_LABEL:-service_name}
      SPAN_LABEL: ${SPAN_LABEL:-span_name}
      STATUS_LABEL: ${STATUS_LABEL:-status_code}
      ERROR_VALUE: ${ERROR_VALUE:-STATUS_CODE_ERROR}
    ports: ["8000:8000"]
    depends_on: [prometheus]
```

3.6 MCP-Server starten

```
cd ~/Documents/otel-cost-workshop/opentelemetry-demo

podman compose --project-name otel-demo \
  --env-file .env --env-file .env.override \
  -f compose.yaml -f compose.observability.yaml \
  -f compose.extras.yaml -f compose.podman.yaml \
  -f compose.mcp.yaml \
  up --build --force-recreate --remove-orphans --detach
```

Logs prüfen:

```
podman compose --project-name otel-demo \
  -f compose.yaml -f compose.observability.yaml \
  -f compose.extras.yaml -f compose.podman.yaml \
  -f compose.mcp.yaml logs --tail 100 mcp-sampling-advisor
```

3.7 MCP ohne KI prüfen

Starte den MCP Inspector:

```
npx -y @modelcontextprotocol/inspector
```

Verbinde den Inspector mit http://localhost:8000/mcp und teste nacheinander: top_operations, error_hotspots, slow_operations, sampling_candidates. Wenn diese Tools keine plausiblen Daten liefern, noch nicht zur KI weitergehen.

3.8 Die KI richtig instruieren

```
Du bist ein Observability Sampling Advisor.
Nutze nur MCP-Daten und erfinde keine Werte.
Erhalte Errors, langsame Traces und geschäftskritische Flows.
Empfehle aggressives Sampling nur bei hohem Volumen,
niedriger Fehlerquote und niedriger Latenz.
Liefere Evidenz, Risiko, Policy-Idee, Validierung und Rollback.
Führe keine Änderung aus; ein Mensch entscheidet.
```

Das konkrete KI-Client-Produkt ist in den beiden Ausgangsdokumenten nicht festgelegt. Deshalb endet die technische Anleitung bewusst an der standardisierten MCP-Schnittstelle und gibt den Prompt vor. Der verwendete KI-Client muss Streamable HTTP MCP unterstützen und auf den lokalen MCP-Endpunkt konfiguriert werden.

3.9 Was du von der KI verlangst

Ein gutes Ergebnis ist keine pauschale Prozentzahl. Verlange pro sichtbarer Operation eine begründete Einordnung. Dieses Format ist eine empfohlene Darstellungsform für deinen Vortrag:

Operation	Volumen	Fehler	p95	Empfehlung	Begründung
service/span	hoch|mittel|niedrig	Wert	Wert	KEEP / KEEP MORE / SAMPLE	Evidenz aus MCP

Danach fragst du die KI sinngemäß: Welche Operationen erzeugen viel Volumen, sind gleichzeitig fehlerarm und schnell? Welche Operationen sollten wegen Fehlern, Latenz oder Business-Kritikalität geschützt werden? Schlage eine Tail-Sampling-Policy vor, aber führe keine Änderung aus.

3.10 KI-Vorschlag in eine Policy übersetzen

Der vorhandene Guide zeigt für geschäftskritische Routen zusätzlich eine string_attribute-Policy. Die dort verwendeten Routen /api/checkout und /api/payment sind ausdrücklich Platzhalter. Verwende nur Attribute/Routen, die du in deinen Traces tatsächlich gesehen und fachlich geprüft hast.

```
tail_sampling:
  policies:
    - name: keep-errors
      type: status_code
      status_code: {status_codes: [ERROR]}
    - name: keep-slow
      type: latency
      latency: {threshold_ms: 1000}
    - name: keep-critical-routes
      type: string_attribute
      string_attribute:
        key: http.route
        values: [/api/checkout, /api/payment]   # NUR PLATZHALTER
    - name: sample-normal
      type: probabilistic
      probabilistic: {sampling_percentage: 10}
```

Jetzt folgt dasselbe Verfahren wie zuvor: Human Review -> Collector-Datei ändern -> Stack neu erstellen -> Logs prüfen -> identischen Traffic -> identische Messungen -> KI-Policy-Spalte ausfüllen.

4. Before/After und Kosten-Story

KPI	Before	After	Ziel
Calls/Spans			reduziert
Bytes, falls messbar			reduziert
Errors sichtbar			ja
Slow Traces sichtbar			ja
Kritische Flows			ja

Reduktion % = (Baseline - After) / Baseline * 100

Für echte Geldbeträge brauchst du zusätzlich die Abrechnungslogik und Vertragspreise des Zielsystems. Ohne diese Daten weist du im Lab die technische Telemetrie-Reduktion aus, nicht eine erfundene Euro-Einsparung.

Story für den Vortrag

1. Wir messen zuerst, statt zu raten. 2. Pauschales Sampling reduziert Daten, kann aber Kontext kosten. 3. Tail Sampling schützt wertvolle Signale. 4. MCP gibt der KI reale Telemetriedaten. 5. Die KI schlägt vor, ein Mensch entscheidet. 6. Danach messen wir erneut und belegen die Reduktion.

5. Rollback

```
cp src/otel-collector/otelcol-config.original.yml \
   src/otel-collector/otelcol-config.yml

podman compose --project-name otel-demo \
  --env-file .env --env-file .env.override \
  -f compose.yaml -f compose.observability.yaml \
  -f compose.extras.yaml -f compose.podman.yaml \
  up --force-recreate --remove-orphans --detach
```

Kontrolliere anschließend erneut Collector-Logs und Jaeger. Damit bist du wieder bei der gesicherten Ausgangskonfiguration.

6. Kurz-Checkliste

☐ Baseline ohne neuen Sampler erfasst
☐ Reproduzierbarer Traffic festgelegt
☐ 50 % konfiguriert, neu gestartet und gemessen
☐ 10 % konfiguriert, neu gestartet und gemessen
☐ Tail Sampling getestet: normal + error + slow
☐ Prometheus-Metriknamen bestätigt
☐ MCP Advisor gebaut und Tools im Inspector geprüft
☐ KI nur mit MCP-Evidenz arbeiten lassen
☐ KI-Policy menschlich geprüft
☐ Before/After mit identischem Test gemessen
☐ Rollback getestet